

EXERCISE - 2ROCK, PAPER & SCISSORS GAME

```

import java.util.Scanner;
public class ROCK-PAPER-SCISSORS-GAME
{
    public static void main(String[] args)
    {
        System.out.println("Taken Input From The Users");
        Scanner sc = new Scanner(System.in);
        System.out.println("Choose 0 for Rock, 1 for PAPER  
and 2 for SCISSOR:");
        int Ram = sc.nextInt();
        int Shyam = sc.nextInt();
        if (Ram == Shyam)
        {
            System.out.println("No one is a Winner, It's  
Draw");
        }
        else if (Ram == 1 && Shyam == 0 || Ram == 2 &&  
Shyam == 1 || Ram == 0 && Shyam == 2)
        {
            System.out.println("Ram is a Winner");
        }
        else
        {
            System.out.println("Shyam is a Winner");
        }
    }
}

```

Chapter → 5

While Loops (Control Instruction)

Sometimes we want our program to execute a few set of instructions over and over again.
 For Example — print 1 to 1000, print multiplication table of 7 etc.

Loops make it easy for us to tell the computer that a given set of instructions need to be executed repeatedly.

★ Types of Loops

Primarily, There are three types of loops in Java.

1. while loop
2. do-while loop
3. for loop

● While Loop

```
while (boolean condition)
```

```
{
```

```
// Statement
```

```
}
```

→ This keeps executing as long as the condition is true.

✗

if the condition never becomes false, the while loop keeps getting executed. Such a loop is called infinite loop.

//code

Write a Program to print natural numbers from 100 to 200.

```

public class WHILE-LOOP
{
    public static void main (String [] args)
    {
        System.out.println ("Enter Natural Numbers:");
        int n = 100;
        while (n <= 200)
        {
            System.out.println (n);
            n++;
        }
        System.out.println ("Finish Using While loops");
    }
}

```

SUN 19

2017

● do-while loop

This loop is similar to a while loop except the fact that it is guaranteed to execute at least once.

```
do {
```

```
    // code
```

```
} while(condition);
```

→ Note: The semicolon.

while → check the condition & execute the code

do-while → execute the code & then checks the condition.

// code

Write a Java program to print n natural numbers using do-while loop.

```
public class DO-WHILE-LOOP
```

```
{
```

```
    public static void main (String[] args)
```

```
    {
```

```
        System.out.println("Enter Natural Number:");
```

```
        int n = 1;
```

```
        do
```

```
        {
```

```
            System.out.println(n);
```

```
            n++;
```

```
        } while(n <= 1000);
```

```
    }
```

```
}
```

• For loop

The syntax of a for loop looks like this:

```
for (Initialize; check-bool-expression; update)
{
    // code;
}
```

A for loop is usually used to execute a piece of code for specific number of times.

//code

Write a Program to print first n odd numbers using a for loop.

```
public class FOR-LOOP
{
    public static void main (String [] args)
    {
        System.out.println("Enter odd numbers:");
        int n = 100;
        for (int i = 0; i < 100; i++)
        {
            System.out.println(2 * i + 1);
        }
    }
}
```

• Decrementing for loop

```

10 for(i = 7; i != 0; i--)
11 {
12     System.out.println(i);
13 }

```

This for loop keeps running until i becomes 0.

2 // Code

3 Write a Program to Print first n natural numbers in Reverse order.

40 public class FOR-LOOP

```

{
    public static void main (String[] args)
    {
        System.out.println("Enter Natural Number:");
        int n = 100;
        for(i = 100; i > 1; i--)
        {
            System.out.println(i);
        }
    }
}

```

★ Break Statement

- The break statement is used to last the loop irrespective of whether the condition is true or false.
- whenever a "break" is encountered inside the loop, the control is sent outside the loop.

//code

```
public class BREAK-STATEMENT
{
    public static void main (String [] args)
    {
        System.out.println ("Using The Loop");
        for (int i = 0; i < 50; i++)
        {
            System.out.println (i);
            System.out.println ("Java is a Great");
            if (i == 2)
            {
                System.out.println ("Ending the loop");
                break;
            }
        }
    }
}
```

★ Continue Statement

- 10 The Continue Statement is used to immediately move to the next iteration of the loop.
- 11 The control is taken to the next iteration thus skipping everything below "continue" inside the loop for that iteration.

//code

```

Public class CONTINUE-STATEMENT
{
    Public static void main (String [] args)
    {
        System.out.println ("Using the Loop");
        for (int i = 0; i < 50; i++)
        {
            if (i == 2)
            {
                System.out.println ("Ending the Loop");
                continue;
            }
            System.out.println (i);
            System.out.println ("Java is Great");
        }
    }
}

```